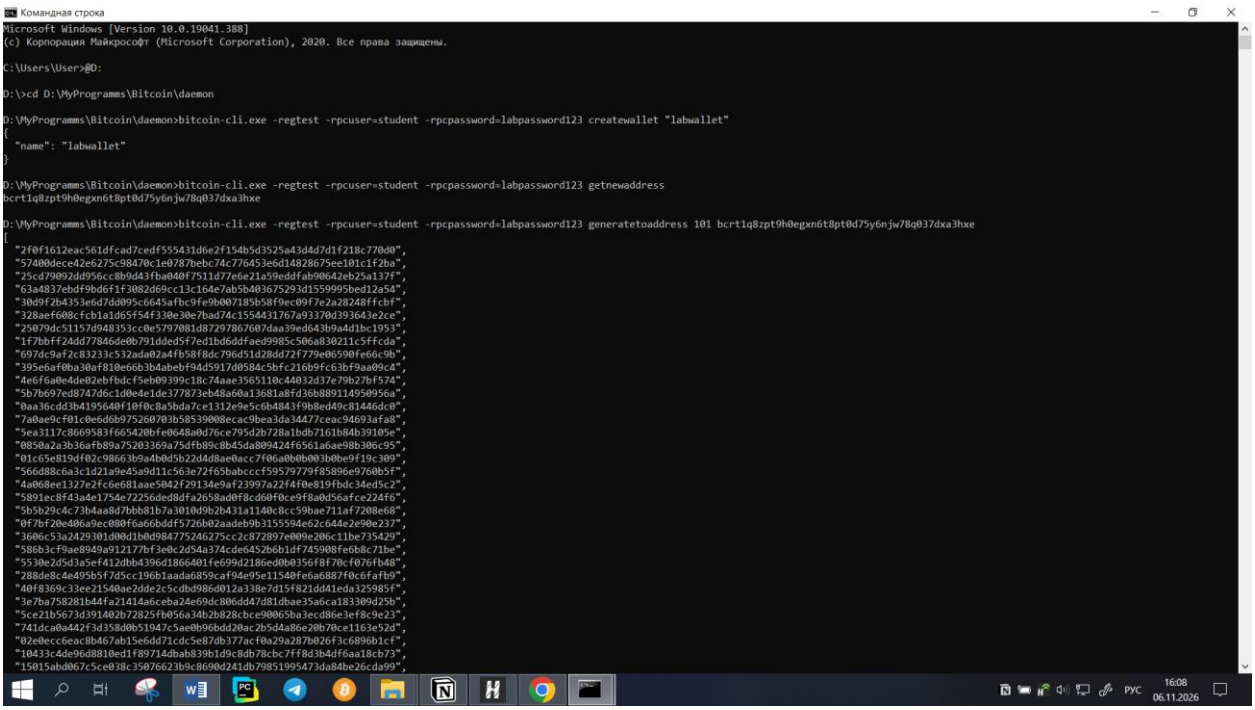
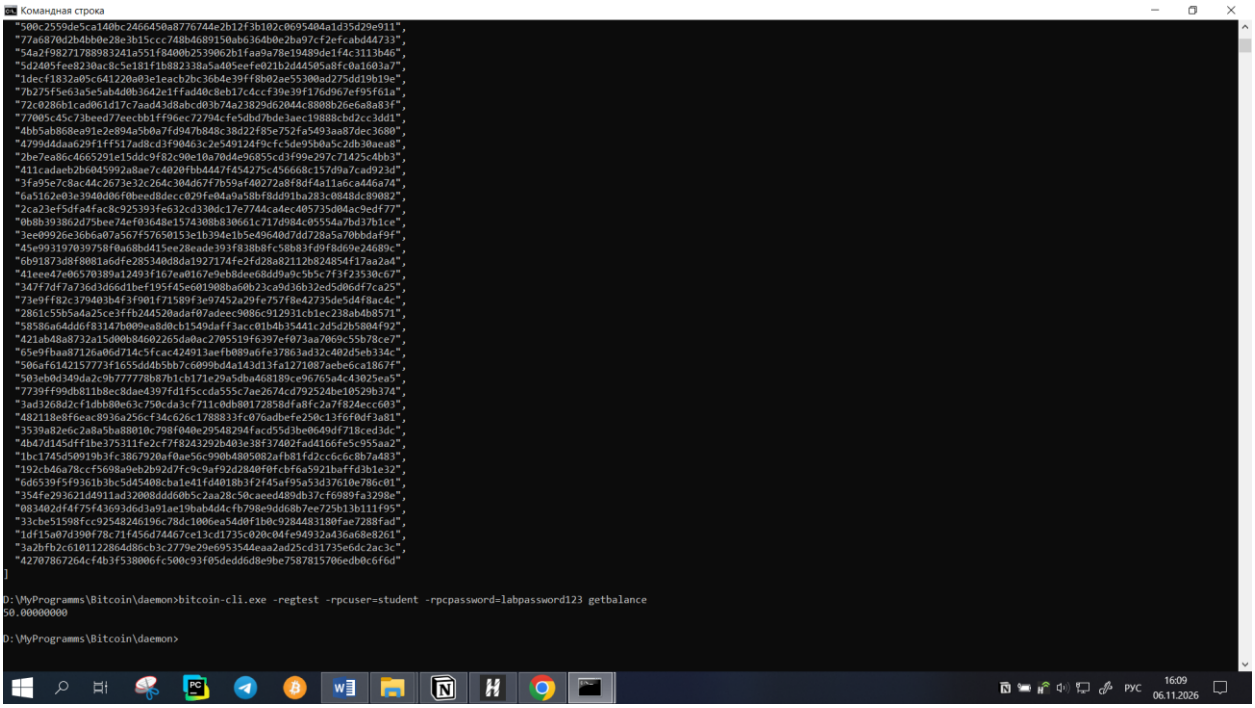
Задание 1.

Была произведена установка и настройка клиента Bitcoin Core для работы в тестовой сети (режим regtest), что позволило развернуть локальный узел блокчейна без необходимости длительной синхронизации и с экономией дискового пространства. Серверная часть узла (bitcoind) была запущена с активированным интерфейсом JSON-RPC, заданными параметрами авторизации пользователя и установленной базовой комиссией. С помощью консольной утилиты bitcoin-cli был создан новый кошелек labwallet и сгенерирован уникальный адрес для получения монет. Для пополнения баланса в тестовой сети была выполнена генерация 101 блока на созданный адрес командой generatetoaddress, поскольку по правилам консенсуса сети Bitcoin вознаграждение за созданный блок становится доступным для траты только после получения ста подтверждений сети. В завершение с помощью команды getbalance была выполнена проверка текущего баланса кошелька, которая подтвердила наличие доступных средств в размере 50 BTC.




Командная строка
"500c2559de5ca140bc2466450a8776744e2b12f3b102c0695404a1d35d29e911",
"77a6870d2b4bb0e28e3b15ccc748b4689150ab6364b0e2ba97cf2efcabd44733",
"54a2f98271788983241a551f8400b2539062b1faa9a78e19489de1f4c3113b46",
"5d2405fee8230ac8c5e181f1b882338a5a405eefe021b2d44505a8fc0a1603a7",
"1decf1832a05c641220a03e1eacb2bc36b4e39ff8b02ae55300ad275dd19b19e",
"7b275f5e63a5e5ab4d0b3642e1ffad40c8eb17c4ccf39e39f176d967ef95f61a",
"72c0286b1cad061d17c7aad43d8abcd03b74a23829d62044c8808b26e6a8a83f",
"77005c45c73beed77eecbb1ff96ec72794cfe5dbd7bde3aec19888cbd2cc3dd1",
"4bb5ab868ea91e2e894a5b0a7fd947b848c38d22f85e752fa5493aa87dec3680",
"4799d4daa629f1ff517ad8cd3f90463c2e549124f9cfc5de95b0a5c2db30aea8",
"2be7ea86c4665291e15ddc9f82c90e10a70d4e96855cd3f99e297c71425c4bb3",
"411cadaeb2b6045992a8ae7c4020fbb4447f45427 5c456668c157d9a7cad923d",
"3fa95e7c8ac44c2673e32c264c304d67f7b59af40272a8f8df4a11a6ca446a74",
"6a5162e03e3940d06f0beed8decc029fe04a9a58bf8dd91ba283c0848dc89082",
"2ca23ef5dfa4fac8c925393fe632cd330dc17e7744ca4ec405735d04ac9edf77",
"0b8b393862d75bee74ef03648e1574308b830661c717d984c05554a7bd37b1ce",
"3ee09926e36b6a07a567f57650153e1b394e1b5e49640d7dd728a5a70bbdaf9f",
"45e993197039758f0a68bd415ee28eade393f838b8fc58b83fd9f8d69e24689c",
"6b91873d8f8081a6dfe285340d8da1927174fe2fd28a82112b824854f17aa2a4",
"41eee47e06570389a12493f167ea0167e9eb8dee68dd9a9c5b5c7f3f23530c67",
"347f7df7a736d3d66d1bef195f45e601908ba60b23ca9d36b32ed5d06df7ca25",
"73e9ff82c379403b4f3f901f71589f3e97452a29fe757f8e42735de5d4f8ac4c",
"2861c55b5a4a25ce3ffb244520adaf07adeec9086c912931cb1ec238ab4b8571",
"58586a64dd6f83147b009ea8d0cb1549daff3acc01b4b35441c2d5d2b5804f92",
"421ab48a8732a15d00b8460226 5da0ac2705519f6397ef073aa7069c55b78ce2",
"65e9fbaa87126a06d714c5fcac424913aefb089a6fe3703ad32c402d5eb334c",
"506af6142157773f1655dd4b5bb7c6099bd4a143d13fa1271087aebe6ca1867f",
"503eb0d349da2c9b777778b87b1cb171e29a5dba468189ce96765a4c43025ea5",
"7739ff99db811b8ec8dae4397fd1f5ccda555c7ae2674cd792524be10529b374",
"3ad3268d2cf1dbb80e63c750cda3cf711c0db80172858dfa8fc2a7f824ecc603",
"482118e8f6eac8936a256cf34c626c1788833fc076adbefe250c13f6f0df3a81",
"3539a82e6c2a8a5ba88010c798f040e29548294facd55d3be0649df718ced3dc",
"4b47d145dff1be375311fe2cf7f8243292b403e38f37402fad4166fe5c955aa2",
"1bc1745d50919b3fc3867920af0ae56c990b4805082afb81fd2cc6c6c8b7a483",
"192cb46a78ccf5698a9eb2b92d7fc9c9af92d2840f0fcbf6a5921baffd3b1e32",
"6d6539f5f9361b3bc5d45408cba1e41fd4018b3f2f45af95a53d37610e786c01",
"354fe293621d4911ad32008ddd60b5c2aa28c50caeed489db37cf6989fa3298e",
"083402df4f75f43693d6d3a91ae19bab4d4cfb798e9dd68b7ee725b13b111f95",
"33cbe51598fcc9254826196c78dc100ea54d0f1b0c928448 3180fae7288fad",
"1df15a07d390f78c71f456d7 467ce13cd1735c020c04fe94932a436a68e8261",
"3a2bfb2c6101122864d86cb3c2779e29e6953544eaa2ad25cd31735e6dc2ac3c",
"42707867264cf4b3f538006fc500c93f05dedd6d8e9be7587815706edb0c6f6d"
]
D:\MyProgramms\Bitcoin\daemon>bitcoin-cli.exe -regtest -rpcuser=student -rpcpassword=labpassword123 getbalance
50.00000000
D:\MyProgramms\Bitcoin\daemon>
РУС
16:09
06.11.2026

Задание 2.

Была разработана программа на языке Python для автоматического подсчета суммы неизрасходованных выходов транзакций (UTXO) для конкретного адреса кошелька посредством обращения к Bitcoin Core JSON-RPC API. Для взаимодействия с локальным узлом использовалась библиотека requests, с помощью которой формировались HTTP POST-запросы с базовой аутентификацией и телом запроса в формате JSON. В основе алгоритма лежит вызов стандартного RPC-метода listunspent, принимающего в качестве параметров диапазон минимального и максимального числа подтверждений, а также массив с целевым адресом для фильтрации результатов. Получив от сервера список подходящих объектов UTXO, программа в цикле обрабатывает каждый неизрасходованный выход, выводит в консоль отладочную информацию (идентификатор транзакции txid, индекс выхода vout и сумму amount) и вычисляет итоговый доступный баланс указанного адреса.

```
Checking UTXOs for address: bcrt1q8zpt9h0egxn6t8pt0d75y6njw78q037dxa3hxe

Found 1 unspent transaction outputs (UTXOs):
  1. TXID: 0d8a23e6d7bb22b0a349aade49c2164db4387500e9e14a85c4344af359eddf64 | vout: 0 | Amount: 50.0 BTC
------------------------------------------------------------
Total Unspent Balance for address: 50.00000000 BTC

Process finished with exit code 0
```

```python
import json
import requests

RPC_URL = "http://127.0.0.1:18443/wallet/labwallet"
RPC_USER = "student"
RPC_PASS = "labpassword123"

MY_ADDRESS = "bcrt1q8zpt9h0egxn6t8pt0d75y6njw78q037dxa3hxe"


def rpc_call(method, params=None):
    headers = {"content-type": "application/json"}
    payload = {
        "jsonrpc": "1.0",
        "id": "lab3_task2",
        "method": method,
        "params": params or [],
    }
    response = requests.post(
        RPC_URL,
        headers=headers,
        data=json.dumps(payload),
        auth=(RPC_USER, RPC_PASS),
    )
    return response.json()


def main():
    print(f"Checking UTXOs for address: {MY_ADDRESS}\n")
```

```
    response = rpc_call("listunspent", [1, 9999999, [MY_ADDRESS]])

    if response.get("error"):
        print("RPC Error:", response["error"])
        return

    utxos = response["result"]

    print(f"Found {len(utxos)} unspent transaction outputs (UTXOs):")

    total_sum = 0.0
    for i, utxo in enumerate(utxos, 1):
        amount = utxo["amount"]
        txid = utxo["txid"]
        vout = utxo["vout"]
        total_sum += amount
        print(f"  {i}. TXID: {txid} | vout: {vout} | Amount: {amount} BTC")

    print("-" * 60)
    print(f"Total Unspent Balance for address: {total_sum:.8f} BTC")


if __name__ == "__main__":
    main()
```

Задание 3.

Была написана программа на языке Python, реализующая полный низкоуровневый цикл создания, криптографической подписи и отправки новой транзакции в сеть через интерфейс JSON-RPC API. На первом этапе скрипт обращается к методу listunspent для выбора доступного неизрасходованного выхода (UTXO), который используется в качестве входа создаваемой транзакции, после чего генерирует новые адреса для получателя перевода и для возврата сдачи с учетом вычета комиссии сети. Далее процесс формирования и отправки транзакции выполняется в три последовательных шага согласно официальной документации Bitcoin Core. Сначала вызывается метод createrawtransaction, который собирает необработанную транзакцию из указанных входов и выходов и возвращает её в виде шестнадцатеричной строки (HEX). Затем метод signrawtransactionwithwallet подписывает полученную сырую транзакцию приватными ключами кошелька, формируя криптографическое доказательство права владения монетами. На заключительном этапе подписанная транзакция передается в метод sendrawtransaction, который транслирует её в сеть (мемпул) и возвращает уникальный идентификатор успешно отправленной транзакции (TXID).

```
[DEBUG] Selected Input UTXO: 0d8a23e6d7bb22b0a349aade49c2164db4387500e9e14a85c4344af359eddf64 (vout: 0), Amount: 50.0
[DEBUG] Recipient Address: bcrt1qayaqgcmu333e8vqquu0r8309qxq34ztdzatnu9
[DEBUG] Change Address:    bcrt1qfct4t6hzaxst8hh2tu0l6vh9453php64u27v3m

1. Created Raw Transaction (HEX):
020000000164dfed59f34a34c4854ae1e9007538b44d16c249deaa49a3b022bbd7e6238a0d0000000000fdffffff0200ca9a3b00000000160014e

2. Signed Transaction (Complete: True):
0200000000010164dfed59f34a34c4854ae1e9007538b44d16c249deaa49a3b022bbd7e6238a0d0000000000fdffffff0200ca9a3b0000000016

============================================================
3. SUCCESS! Transaction Broadcasted. TXID:
66e0b791f60d2b31aafcc7be8f67db2c5e07ec3d348cb9f9d9d9517c827047d3
============================================================

Process finished with exit code 0
```

```python
import json
import requests

RPC_URL = "http://127.0.0.1:18443/wallet/labwallet"
RPC_USER = "student"
RPC_PASS = "labpassword123"


def rpc_call(method, params=None):
    headers = {"content-type": "application/json"}
    payload = {
        "jsonrpc": "1.0",
        "id": "lab3_task3",
        "method": method,
```

```
        "params": params or [],
    }
    response = requests.post(
        RPC_URL,
        headers=headers,
        data=json.dumps(payload),
        auth=(RPC_USER, RPC_PASS),
    )
    res_json = response.json()
    if res_json.get("error"):
        raise Exception(f"RPC Error in {method}: {res_json['error']}")
    return res_json["result"]


def main():
    utxos = rpc_call("listunspent")
    if not utxos:
        print("No UTXOs available to spend!")
        return

    selected_utxo = utxos[0]
    txid_in = selected_utxo["txid"]
    vout_in = selected_utxo["vout"]
    input_amount = selected_utxo["amount"]
    print(
        f"[DEBUG] Selected Input UTXO: {txid_in} (vout: {vout_in}), Amount:
{input_amount} BTC"
    )

    recipient_address = rpc_call("getnewaddress")
    change_address = rpc_call("getnewaddress")
    print(f"[DEBUG] Recipient Address: {recipient_address}")
    print(f"[DEBUG] Change Address:    {change_address}")

    send_amount = 10.0
    fee = 0.0001
    change_amount = round(input_amount - send_amount - fee, 8)

    inputs = [{"txid": txid_in, "vout": vout_in}]
    outputs = {recipient_address: send_amount, change_address: change_amount}

    raw_tx = rpc_call("createrawtransaction", [inputs, outputs])
    print(f"\n1. Created Raw Transaction (HEX):\n{raw_tx}\n")

    signed_tx_obj = rpc_call("signrawtransactionwithwallet", [raw_tx])
    signed_hex = signed_tx_obj["hex"]
    print(
        f"2. Signed Transaction (Complete:
{signed_tx_obj['complete']}):\n{signed_hex}\n"
    )

    broadcasted_txid = rpc_call("sendrawtransaction", [signed_hex])
    print("=" * 60)
    print(f"3. SUCCESS! Transaction Broadcasted. TXID:\n{broadcasted_txid}")
    print("=" * 60)


if __name__ == "__main__":
    main()
```